Greedy Technique

Asst. Prof. Dr. Aurawan Imsombut

https://kasets.art/kxXMOS

Outline

- Greedy Technique
- Change-Making Problem
- Knapsack Problem
- Scheduling Problem
- Huffman codes

Greedy Technique

- Constructs a solution to an optimization problem piece by piece through a sequence of choices that are:
  - feasible
  - locally optimal
  - irrevocable
- For some problems, yields an optimal solution for every instance.
- For most, does not but can be useful for fast approximations.

Applications of the Greedy Strategy

- Optimal solutions:
  - change making for "normal" coin denominations
  - minimum spanning tree (MST)
  - single-source shortest paths
  - simple scheduling problems
  - Huffman codes
- Approximations:
  - traveling salesman problem (TSP)
  - knapsack problem
  - other combinatorial optimization problems

Change-Making Problem

- Given unlimited amounts of coins of denominations $d_1 > \ldots > d_m$, give change for amount n with the least number of coins

Example: d_1 = 10THB, d_2 = 5THB, d_3 = 2THB, d_4 = 1THB
and n = 48THB

Greedy solution: $4d_1 + 1d_2 + 1d_3 + 1d_4$ => optimal solution

Assume: d_1 = 10THB, d_2 = 5THB, d_3 = 4THB, d_4 = 2THB , d_5 = 1THB
and n = 48c

Greedy solution: $4d_1 + 1d_2 + 1d_4 + 1d_5$ => not optimal solution

Optimal solution: $4d_1 + 2d_3$

Change-Making Problem

Greedy solution is

- optimal for any amount and "normal" set of denominations
- may not be optimal for arbitrary coin denominations

Knapsack Problem

- Given a set of items having some weight and value/profit associated with it.
- The knapsack problem is to find the set of items such that the total weight is less than or equal to a given limit (size of knapsack) and the total value/profit earned is as large as possible
- Knapsack problem has two variants.
  - Binary or 0/1 knapsack : Item cannot be broken down into parts.
  - Fractional knapsack : Item can be divided into parts.

https://codecrucks.com/fractional-knapsack-using-greedy-algorithm/

Knapsack Problem

- Knapsack problem can be formulated as follow :

  Maximize $\sum_{i=1}^{n} v_i x_i$ subjected to $\sum_{i=1}^{n} w_i x_i \leq M$

  - $x_i \in (0, 1)$ for binary knapsack
  - $x_i \in [0, 1]$ for fractional knapsack

  - Let
    - $X = < x_1, x_2, x_3, \ldots\ldots, x_n >$ is the set of n items.
    - $W = < w_1, w_2, w_3, \ldots, w_n >$ is the set of weight of each items
    - $V = < v_1, v_2, v_3, \ldots, v_n >$ is the set of value associated with each items
    - M is the knapsack capacity.

Knapsack Problem

- Example: M = 15

Item (x_i)	Value (v_i)	Weight (w_i)
1	12	6
2	5	2
3	14	7
4	15	5

- **Binary knapsack:**
  Selecting the solution with respect to the maximum value/weight

Item (x_i)	Value (v_i)	Weight (w_i)	v_i/w_i
4	15	5	3
2	5	2	2.5
3	14	7	2
1	12	6	2

Selected Item	Total Value	Total Weight $\leq M$
4	15	5
4, 2	15+5 = 20	5+2 = 7
4, 2, 3	**20 + 14 = 34**	**7 + 7 = 14**

Knapsack Problem

- Example: M = 15

Item (x_i)	Value (v_i)	Weight (w_i)
1	12	6
2	5	2
3	14	7
4	15	5

- **Fractional knapsack**

Item (x_i)	Value (v_i)	Weight (w_i)	v_i/w_i
4	15	5	3
2	5	2	2.5
3	14	7	2
1	12	6	2

Selected Item	Total Value	Total Weight $\leq M$
4	15	5
4, 2	15+5 = 20	5+2 = 7
4, 2, 3	20 + 14 = 34	7 + 7 = 14
4, 2, 3, 1*	**34 + 2.0 = 36**	**14 + 1 = 15**

* Select item 1 = $\frac{15-14}{6} = \frac{1}{6} = 0.1667$

value of selected item 1 = 12 * 0.1667 = 2.004

Knapsack Problem

- **Fractional knapsack algorithm**

```
Greedy Knapsack (A[1..n], M) {
    sort A by v/w in decrease order
    sumWeight = 0
    sumValue = 0
    for (i=1; i <= n AND sumWeight <= M; i++){
        if((sumWeight + A[i].w) <= M)
            A[i].x = 1
        else
            A[i].x = (M – sumWeight) / A[i].w
        sumWeight += A[i].x * A[i].w
        sumValue += A[i].x * A[i].v
    }
    return A
}
```

Exercise

1. For the given set of items and knapsack capacity = 60 kg, find the optimal solution for the fractional knapsack problem making use of greedy approach.

Item	Weight	Value
1	5	30
2	10	40
3	15	45
4	22	77
5	25	90

Exercise

2. Design a greedy algorithm for the assignment problem. There are n people who need to be assigned to n jobs, one person per job. The cost of assigning person i to job j is $C[i, j]$.

   Find an assignment that minimizes the total cost.

Example

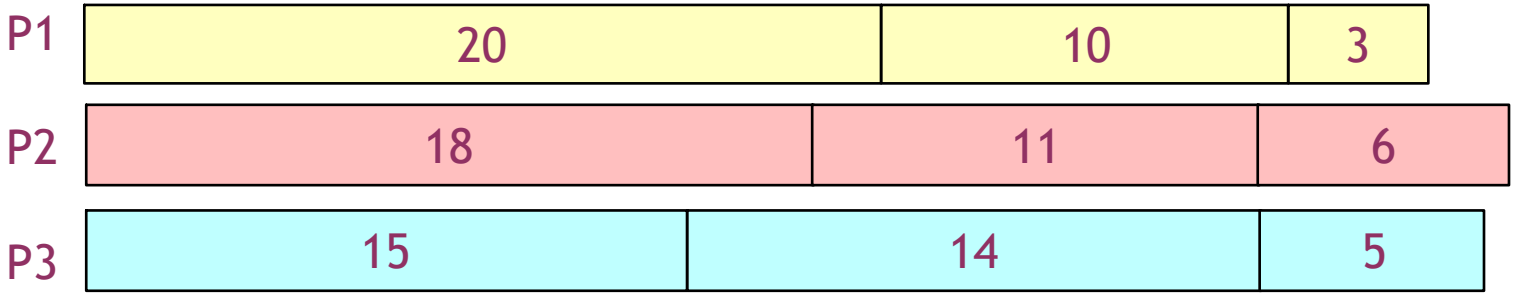
	Job 1	Job 2	Job 3	Job 4
Person 1	**9**	**2**	**7**	**8**
Person 2	**6**	**4**	**3**	**7**
Person 3	**5**	**8**	**1**	**8**
Person 4	**7**	**6**	**9**	**4**

Does your greedy algorithm always yield an optimal solution?

A Scheduling Problem

MODIFIED GREEDY MULTIPROCESSOR SCHEDULING:
Sort the jobs in decreasing order of processing times
For $i = 1$ to n do
  Assign Job J_i to the machine with *least current load*
  Update load of the machine that receives job J_i

- You have to run nine jobs, with running times of 3, 5, 6, 10, 11, 14, 15, 18, and 20 minutes.
- You have three processors on which you can run these jobs.
- You decide to do the longest-running jobs first, on whatever processor is available.

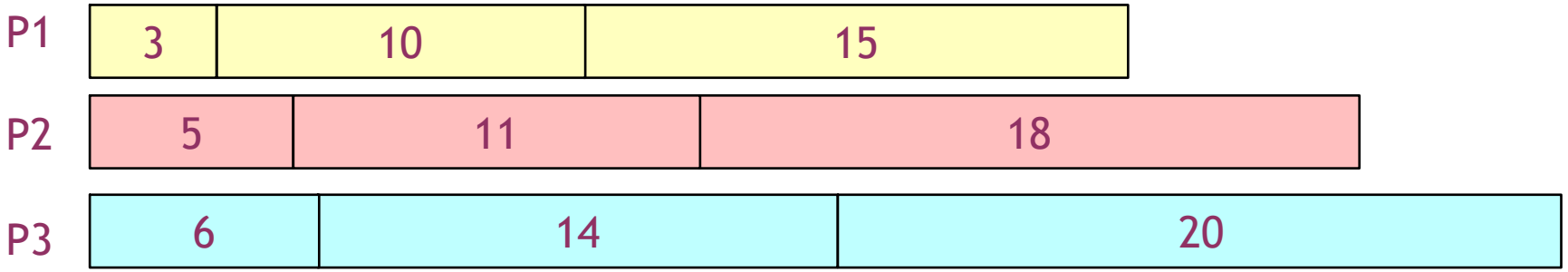


- Time to completion: 18 + 11 + 6 = 35 minutes
- This solution isn't bad, but we might be able to do better

Another Approach

GREEDY MULTIPROCESSOR SCHEDULING:
Order (list) the jobs arbitrarily
For $i = 1$ to n do
  Assign Job J_i to the machine with *least current load*
  Update load of the machine that receives job J_i

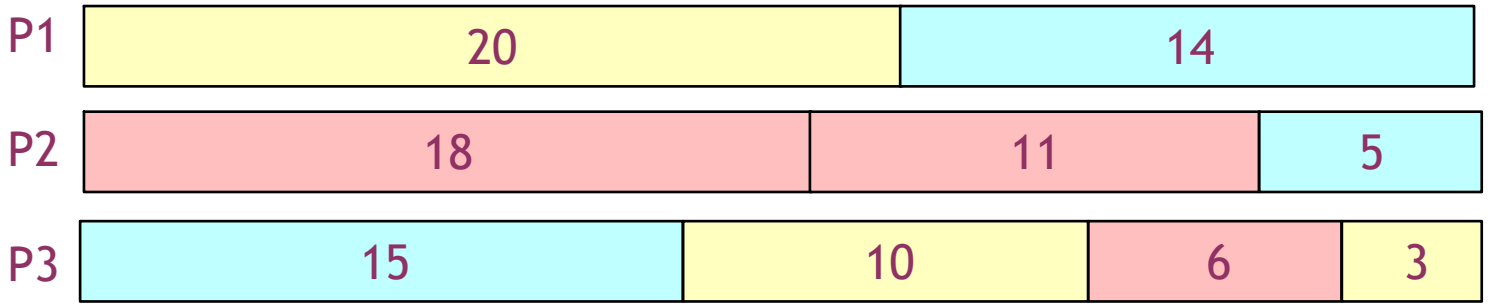
- What would be the result if you ran the *shortest* job first?
- Again, the running times are 3, 5, 6, 10, 11, 14, 15, 18, and 20 minutes



- That wasn't such a good idea; time to completion is now 6 + 14 + 20 = 40 minutes
- Note, however, that the greedy algorithm itself is fast
  - All we had to do at each stage was pick the minimum or maximum

An Optimum Solution

- Better solutions do exist:



  - This solution is clearly optimal (why?)
  - Clearly, there are other optimal solutions (why?)
  - How do we find such a solution?
    - One way: Try all possible assignments of jobs to processors
    - Unfortunately, this approach can take exponential time

Minimizing the maximum completion time (makes pan) problem is NP-hard.

Coding Problem

- *Coding*: assignment of bit strings to alphabet characters
- *Codewords*: bit strings assigned for characters of alphabet

- Two types of codes:
  - *fixed-length encoding* (e.g., ASCII)
  - *variable-length encoding* (e,g., Morse code)

- *Prefix-free codes*: no codeword is a prefix of another codeword

- Problem: If frequencies of the character occurrences are known, what is the best binary prefix-free code?

Huffman codes

- Developed by David, A. Huffman in the 1952 paper "A method for the Construction of Minimum Redundancy Codes"
- Each code is a binary string that is used for transmission of the corresponding message.
- Example:

BAGGAGE ROOM using ASCII code (every character using 7 bits)

B(1000010)	A(1000001)	G(1000111)	G(1000111)
A(1000001)	G(1000111)	E(1000101)	space(0100000)
R(1010010)	O(1001111)	O(1001111)	M(1001101) => 7 * 12 = 84 bits

BAGGAGE ROOM using Huffman Code

B(0100) A(101) G(11) G(11) A(101) G(11) E(0101)
space(1001) R(011) O(00) O(00) M(1000) => 35 bits

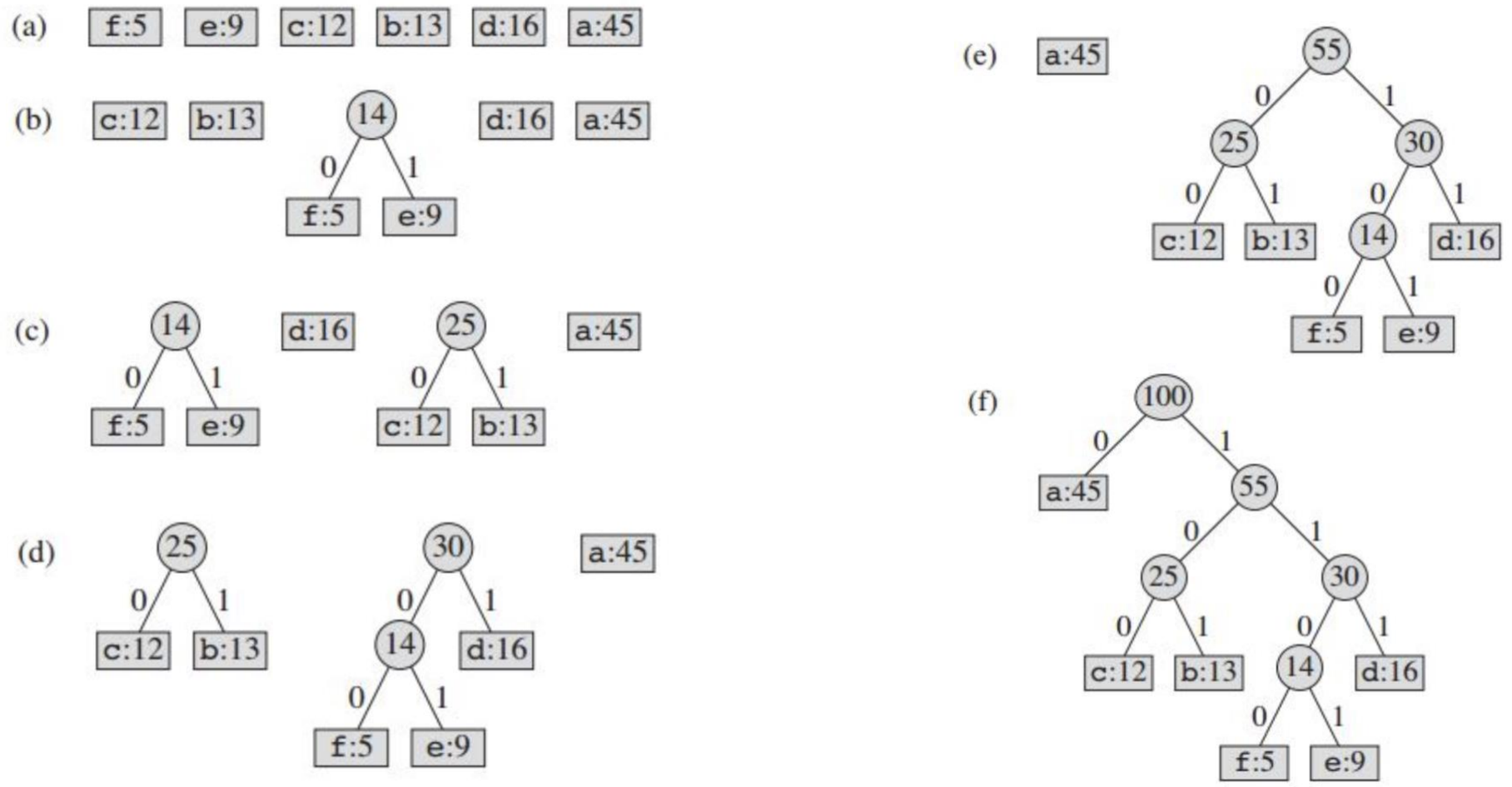
Huffman codes

- Any binary tree with edges labeled with 0's and 1's yields a prefix-free code of characters assigned to its leaves
- Optimal binary tree minimizing the expected (weighted average) length of a codeword can be constructed as follows

- Huffman's algorithm
- Initialize n one-node trees with alphabet characters and the tree weights with their frequencies.
- Repeat the following step n-1 times: join two binary trees with smallest weights into one (as left and right subtrees) and make its weight equal the sum of the weights of the two trees.
- Mark edges leading to left and right subtrees with 0's and 1's, respectively.

Example

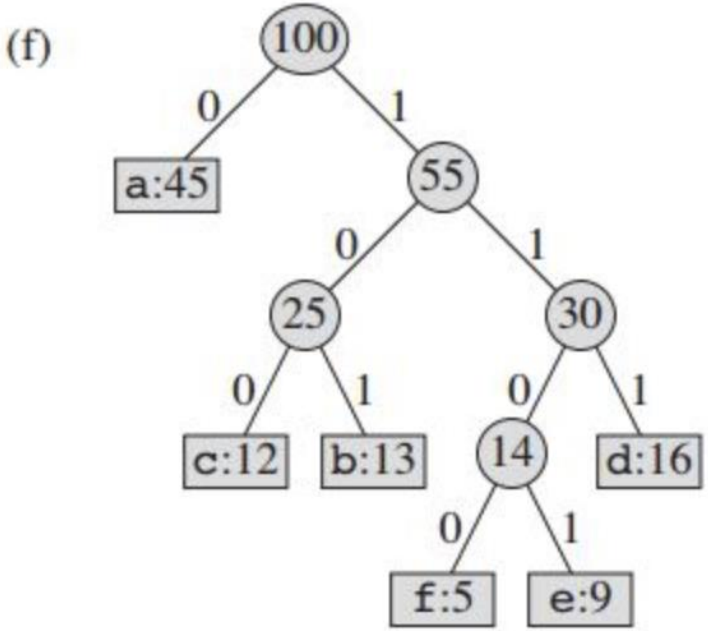
Character	a	b	c	d	e	f
Frequency (in thousands)	45	13	12	16	9	5



https://riptutorial.com/algorithm/example/23995/huffman-coding

Example:

Alphabets	a	b	c	d	e	f
Code word	0	101	100	111	1101	1100
Frequency	45	13	12	16	9	5

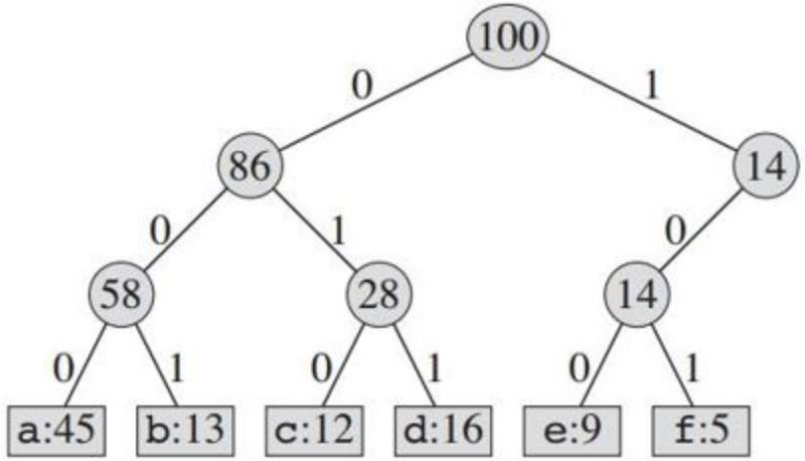


Average bits per character:

$$= \frac{(1*45)+(3*13)+(3*12)+(3*16)+(4*9)+(4*5)}{100}$$

= 2.24

For fixed-length encoding = 3
($2^3 = 8$ character)



https://riptutorial.com/algorithm/example/23995/huffman-coding

Huffman codes

- Time Complexity: $O(n \log n)$ each iteration requires $O(n \log n)$ time to determine the smallest weight
  and there would be $O(n)$ iterations.

```
HUFFMAN(C)
 n = |C|
 Q = C
 for I = 1 to n-1
    allocate a new node z
    z.left = x = EXTRACT-MIN(Q)
    z.right = y = EXTRACT-MIN(Q)
    z.freq = x.freq + y.freq
    INSERT(Q, z)
 return EXTRACT-MIN(Q) // return the root of the tree
```

Other Greedy Algorithms

- **Dijkstra's** algorithm for finding the shortest path in a graph
  - Always takes the *shortest* edge connecting a known node to an unknown node
- **Kruskal's** algorithm for finding a minimum-cost spanning tree
  - Always tries the *lowest-cost* remaining edge
- **Prim's** algorithm for finding a minimum-cost spanning tree
  - Always takes the *lowest-cost* edge between nodes in the spanning tree and nodes not yet in the spanning tree

Exercise

1. CopyMachine
2. Pair of Character
3. Minimum Coin
4. Oasis
5. Square Coin Change
6. AnnLikeEqual
7. Fractional Knapsack
8. FrogCrossing
9. Huffman Code

Exercise

- 10026 Shoemaker's problem
  https://onlinejudge.org/index.php?option=com_onlinejudge&Itemid=8&page=show_problem&problem=967#google_vignette

- 11150 Cola
  https://onlinejudge.org/index.php?option=com_onlinejudge&Itemid=8&page=show_problem&problem=2091

- 11389 Bus Driver
  https://onlinejudge.org/index.php?option=com_onlinejudge&Itemid=8&page=show_problem&problem=2384

- 11567 - Moliu Number Generator
  https://onlinejudge.org/index.php?option=com_onlinejudge&Itemid=8&page=show_problem&problem=2614